VELLORE INSTITUTE OF TECHNOLOGY

Vellore Campus

DIGITAL ASSIGNMENT 1

CPU Scheduling Simulator with Gantt Charts and Comparison Views

Project Survey Document

Submitted by	Anubhav Kayal
Registration Number	24BCE2204
Subject	Operating Systems
Faculty	Dr. Niha K Basha

Vellore Institute of Technology, Vellore

Contents

1. Project title
2. Introduction
3. Problem statement
4. Objectives
5. Survey on existing work
6. Existing and proposed systems
7. Methodology with flowchart
8. Software and hardware requirements
9. System architecture
10. Implementation timeline
11. Expected outcomes
12. Future scope
13. Challenges and ethical considerations
14. Conclusion

References

1. Project title

CPU Scheduling Simulator with Gantt Charts and Comparison Views.

The project is a desktop program written in Python. It simulates six CPU scheduling policies, draws the result as a Gantt chart, and lets the user compare all six on the same set of processes.

2. Introduction

A computer usually has more runnable processes than CPU cores. The operating system has to decide which process runs next and for how long. The part of the kernel that makes that decision is the CPU scheduler, and the rule it follows is the scheduling algorithm.

The choice of rule changes how the system feels. A rule that favours short jobs keeps the average waiting time low but can leave a long job waiting for a long time. A rule that gives every process a fixed slice of time responds quickly to everyone but pays for it in extra context switches and a longer average turnaround. Textbooks explain these differences with small worked examples drawn on paper as Gantt charts. That works, but it is slow to redo by hand whenever the numbers change.

This project turns those paper exercises into a program. The user types in a few processes with their arrival and burst times, picks an algorithm, and gets the Gantt chart and the waiting, turnaround and response times straight away. A second screen runs every algorithm on the same

input and plots the averages next to each other.

What the program covers

The simulator implements First Come First Served (FCFS), Shortest Job First without preemption (SJF), Shortest Remaining Time First (SRTF, the preemptive version of SJF), Priority scheduling with and without preemption, and Round Robin with a quantum chosen by the user. It is a simulation. It does not schedule real operating system processes. It works on the numbers the user enters.

3. Problem statement

Students learn scheduling from static diagrams and a few solved examples. When the arrival times, burst times or priorities change, the whole chart has to be drawn again by hand. Mistakes creep in, especially with preemptive algorithms such as SRTF and Round Robin, where one process is cut into several pieces on the timeline. It is also hard to see by hand how one workload behaves under six different rules.

The problem this project addresses is the lack of a small, accurate tool that runs these algorithms on any input, shows the exact execution order, reports the standard metrics, and compares the algorithms on one workload without the user redoing any arithmetic.

4. Objectives

The first objective is to implement six scheduling policies correctly: FCFS, SJF, SRTF, Priority (non-preemptive and preemptive) and Round Robin. Correct here means that the same input always gives the same schedule, including when two processes tie.

The second is to report the average waiting time, average turnaround time and average response time for each run, along with CPU utilisation and throughput.

The third is to draw the schedule as a Gantt chart inside the application window, with idle gaps shown as their own blocks, and to list the per-process numbers in a table under it.

The fourth is to provide a comparison screen that runs all algorithms on the same workload and shows which one gives the lowest average waiting time.

The fifth is to keep the scheduling code separate from the interface and cover it with automated tests, so that results can be checked without opening the GUI.

5. Survey on existing work

Scheduling has been studied and taught for a long time, so the survey below is split into three parts: how textbooks present it, how real systems and research treat it, and what student-level simulators already exist.

5.1 Textbooks and teaching material

Silberschatz, Galvin and Gagne, in *Operating System Concepts*, present FCFS, SJF, priority scheduling, Round Robin and multilevel queues. They explain each with a Gantt chart and a short table of waiting times. This is the style of example the simulator reproduces. Tanenbaum's *Modern Operating Systems* covers the same algorithms and groups them by system type: batch, interactive and real-time. Arpaci-Dusseau and Arpaci-Dusseau, in *Operating Systems: Three Easy Pieces*, introduce turnaround time and response time as the two main metrics and then move on to multi-level feedback queues and proportional-share scheduling. The book is free to read online from the University of Wisconsin-Madison.

All three treat the algorithms on paper. None of them is an interactive tool, which is the gap a simulator fills.

5.2 Real systems and research

C. L. Liu and James W. Layland published "Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment" in the *Journal of the ACM*, volume 20, issue 1, pages 46 to 61, in January 1973. The paper is the usual source for rate-monotonic scheduling, which uses fixed priorities, and earliest-deadline-first scheduling, which changes priorities by deadline. It shows that a fixed-priority scheduler may only be able to use about 70 percent of the processor for large task sets, while deadline-based priorities can reach full use.

The Linux Completely Fair Scheduler, written by Ingo Molnar and merged in kernel 2.6.23, takes a different route. The kernel documentation says its design is based on modelling an ideal, precise multi-tasking CPU, and it tracks a virtual runtime for each task so that tasks get an equal share over time. The same documentation notes that CFS is now being replaced by the EEVDF scheduler. Real-time policies in Linux (SCHED_FIFO and SCHED_RR) follow POSIX and are separate from CFS.

These systems are far more complicated than the classroom algorithms. They handle many cores, thousands of tasks, caches and deadlines. The six algorithms in this project are the simpler ancestors that the textbooks use to explain the ideas.

5.3 Student-level simulators

A quick search on GitHub shows that many people have built scheduling simulators. I looked at the descriptions only and did not run them. One project, OS algorithms visualization by Manas-Maahir, is a browser tool in HTML, CSS and JavaScript with FCFS, SJF, priority, Round Robin and multilevel queues, and an animated Gantt chart with play and pause. Azriy-Nazariee's CPU-Scheduling is a Python program using Tkinter and Matplotlib with Round Robin, SJF and non-preemptive priority. Module4Assign3 by kitzki is a C++ command-line program that prints a Gantt chart, waiting and turnaround times, CPU utilisation and throughput. SchedSim by def3r is another C++ tool that covers SJF, Round Robin and shortest remaining time. Rohit Kaushal hosts a web simulator that plots a Gantt chart for each algorithm and a comparison graph.

Most of these cover a subset of the algorithms, or run only in a browser or only on the command line. The ones that combine a desktop window, a Gantt chart and a comparison graph are less common, and the ones I could read about did not mention unit tests.

6. Existing and proposed systems

Existing approach

The usual way to study these algorithms is to work them out on paper or in a spreadsheet. A student draws the timeline, counts the waiting time of each process, and averages. For an algorithm such as Round Robin with five or six processes, this takes several minutes and a single slip in the ready-queue order changes every number after it. Command-line programs and web simulators help, but they often leave out some algorithms, hide the tie-breaking rule, or show averages without a way to compare algorithms on one input.

Proposed system

The proposed system is a Tkinter desktop application. The user adds processes in a form, selects an algorithm and presses Run Simulation. The application copies the processes, runs the chosen scheduler, and draws the Gantt chart with matplotlib inside the window. A results table lists arrival, burst, start, completion, waiting, turnaround and response time for every process, with the averages on the last row. A status line below it shows the average waiting time, average turnaround time and CPU utilisation.

The Compare All Algorithms tab runs all six policies on the same input and draws a grouped bar chart of the average waiting and turnaround times. A line under the chart names the algorithm with the lowest average waiting time.

Two design decisions matter for correctness. The first is a fixed tie-breaking rule: when two processes are equal on the deciding value, the lower PID goes first, and in priority scheduling equal priorities fall back to arrival time and then PID. The second is that idle time is recorded as an explicit IDLE block, so a gap between two processes shows up on the chart instead of being hidden.

7. Methodology with flowchart

The project was built in the order shown in the next section: data model first, then each algorithm with its tests, then the metrics, and last the interface. Writing the algorithms before the interface meant that each one could be checked against hand-worked answers before any drawing code existed.

How each algorithm decides

FCFS picks the process that arrived first. SJF picks, from the processes that have already arrived, the one with the shortest burst, and lets it finish. SRTF does the same check every time unit and switches if a newly arrived process has less time remaining than the one running. Priority scheduling picks the process with the best priority number, where a lower number means higher priority by default, and the preemptive version re-checks whenever a process arrives. Round Robin keeps a ready queue and gives each process at most one quantum before moving it to the back. When a new process arrives during a quantum, it is queued ahead of the process that was just preempted.

Metrics

Turnaround time is completion time minus arrival time. Waiting time is turnaround time minus burst time. Response time is the time of the first CPU slice minus arrival time. CPU utilisation is the busy time divided by the total timeline length. Throughput is the number of finished processes divided by the total time.

Flow of one run

Figure 1 shows the sequence the program follows from the moment the user enters data until the results are on the screen.

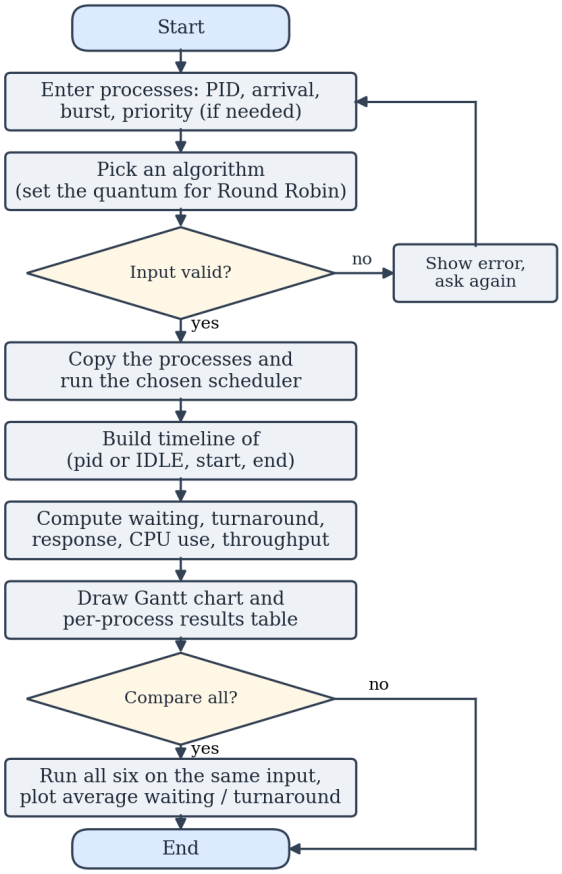


Figure 1. Flowchart of one simulation run and the optional comparison.

Input is checked before anything runs. PIDs must be unique integers, arrival times cannot be negative, burst times must be positive, and Round Robin needs a positive whole-number quantum. A bad value produces a message box and nothing is simulated.

8. Software and hardware requirements

Software

Python 3.11 or newer with Tkinter available (the `_tkinter` module). matplotlib 3.7 or newer for the charts. pytest 7.4 or newer to run the tests. No other libraries are used. Git was used for version control and the code is hosted at github.com/anubhav-kayal/os-lab-project.

On macOS the Python installed through pyenv or Homebrew is sometimes built without Tcl/Tk, which makes the GUI fail with "No module named _tkinter". The project handles this in two ways. The program prints a message explaining what to do, and a script called `run.sh` looks for a Python that does have Tkinter and starts the application with it. The python.org Framework build of Python 3.12 works.

Hardware

Any computer that can run Python 3.11 and show a desktop window will do. The simulator does very little work: the largest job is drawing a chart. I developed and tested it on a Mac laptop running macOS and did not measure memory use, so I will not quote a number.

9. System architecture

The code is split into a core that knows nothing about windows and a set of GUI modules that draw the results. Figure 2 shows how the pieces connect.

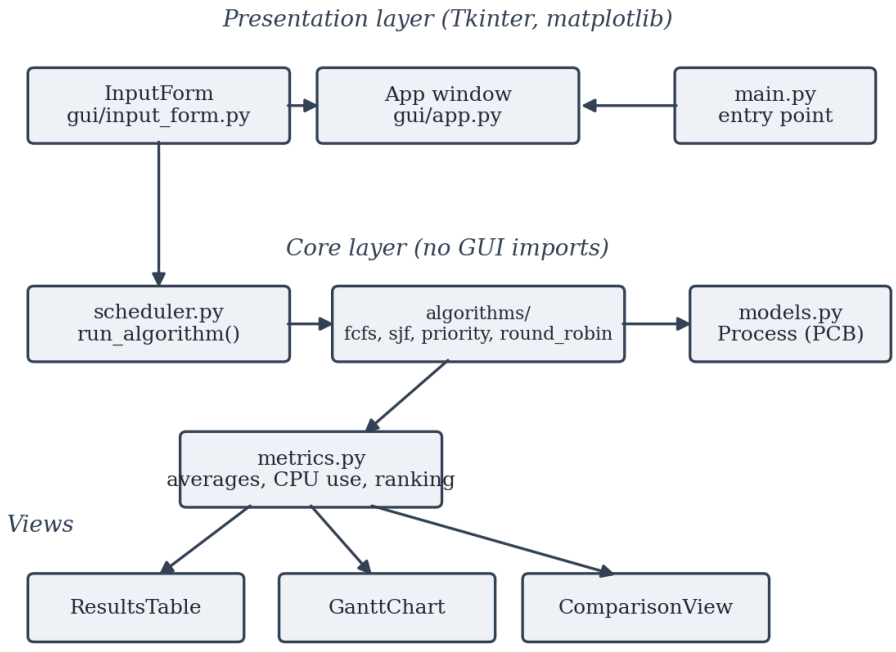


Figure 2. Module layout. The core layer has no GUI imports and can be tested on its own.

`models.py` holds the Process class, which acts as a small process control block with the arrival, burst and priority values and the fields that the schedulers fill in: start, completion, waiting, turnaround and response time. It checks its own values when created.

Each file in `algorithms/` implements one family of schedulers. Every one of them has the same return shape, a pair made of the finished process list and a timeline of (pid or IDLE, start, end) segments. They work on copies, so the user's input is never changed. `scheduler.py` turns the name picked in the dropdown into the right function call. `metrics.py` computes the averages, CPU utilisation, throughput and the ranking used by the comparison screen.

In `gui/`, `input_form.py` builds the entry form and validation, `gantt_chart.py` draws the timeline with matplotlib, `comparison_view.py` draws the bar chart, and `app.py` puts them together in one window with two tabs. The source under `src/` is about 1,700 lines of Python.

10. Implementation timeline

The work was done in thirteen numbered phases, each on its own Git branch and merged into main with a commit message beginning "Phase" and the phase number. All of the commits are dated 31 July.

Phase 0 set up the folders, requirements file and repository. Phase 1 wrote the Process model. Phases 2 to 5 added FCFS, then SJF with SRTF, then Priority in both modes, then Round Robin, each with its unit tests. Phase 6 added the metrics module. Phase 7 built the input form. Phase 8 added the Gantt chart. Phase 9 added the results table and the comparison dashboard. Phase 10 handled edge cases such as an empty list, duplicate PIDs and idle gaps, and brought the full test suite to green. Phase 11 wrote the README, the sample test cases and a report template. Phase 12 was a final review against a pre-submission checklist.

Two small fixes followed. One makes the program print clear instructions when Tkinter is missing, and the other marks `run.sh` as executable. The last commit adds the screenshots used in this document.

11. Expected outcomes

The program should give the same schedule every time for a given input, and that schedule should match a hand-worked answer. To check this I ran one workload through every algorithm. The processes are P1 (arrival 0, burst 5), P2 (arrival 1, burst 3) and P3 (arrival 2, burst 4). For the priority runs P1 and P3 have priority 1 and P2 has priority 2. Round Robin used a quantum of 2. The figures below are screenshots of the running application.

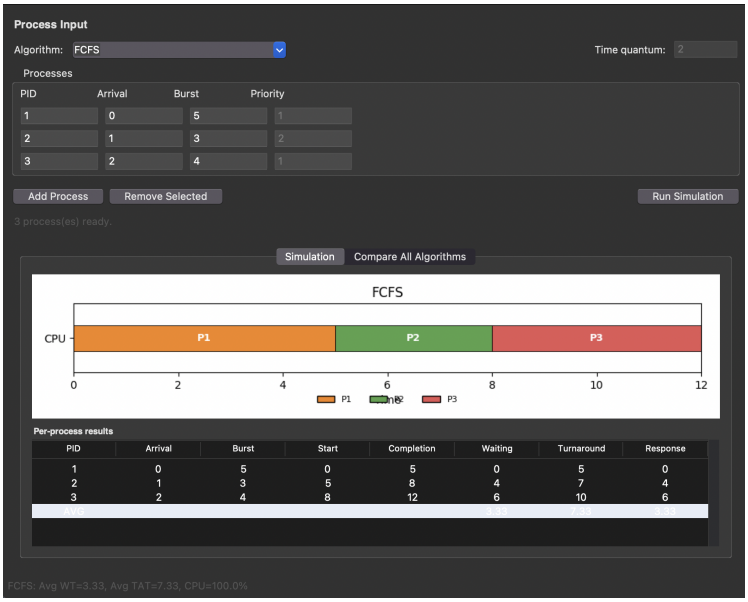


Figure 3. FCFS. P1 runs from 0 to 5, P2 from 5 to 8 and P3 from 8 to 12. Average waiting time 3.33, average turnaround 7.33.

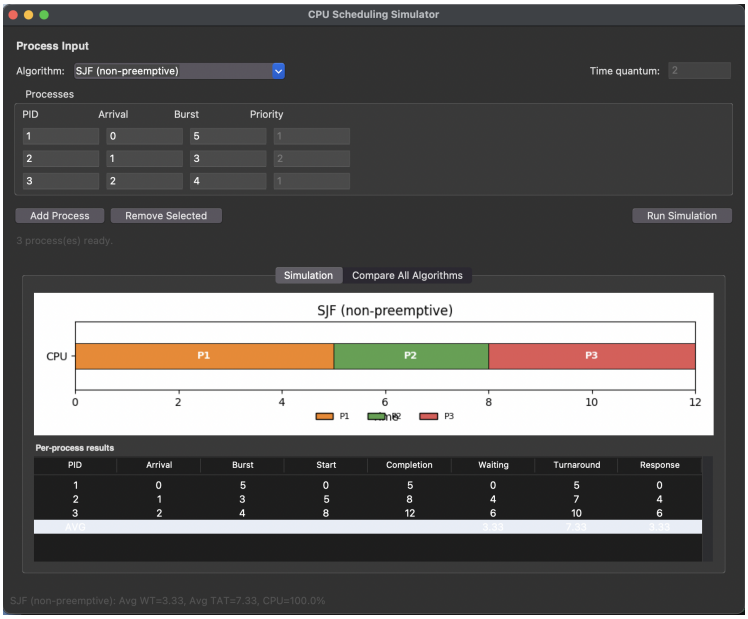


Figure 4. SJF without preemption. P1 has already started when the others arrive, so it finishes first. P2 has the shorter burst and goes before P3. The averages match FCFS for this input.

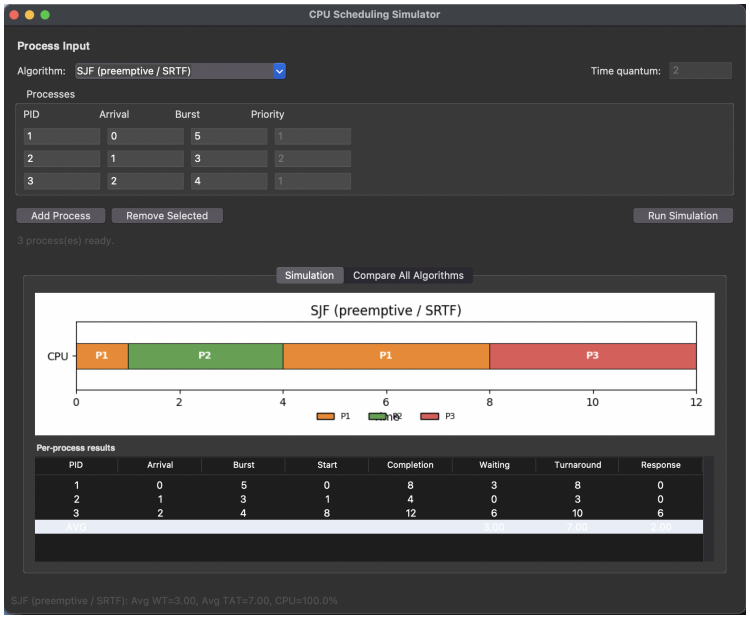


Figure 5. SRTF. P2 arrives at time 1 with less work left than P1, so P1 is paused after one unit and resumes at time 4. Average waiting time 3.00, turnaround 7.00, response 2.00.

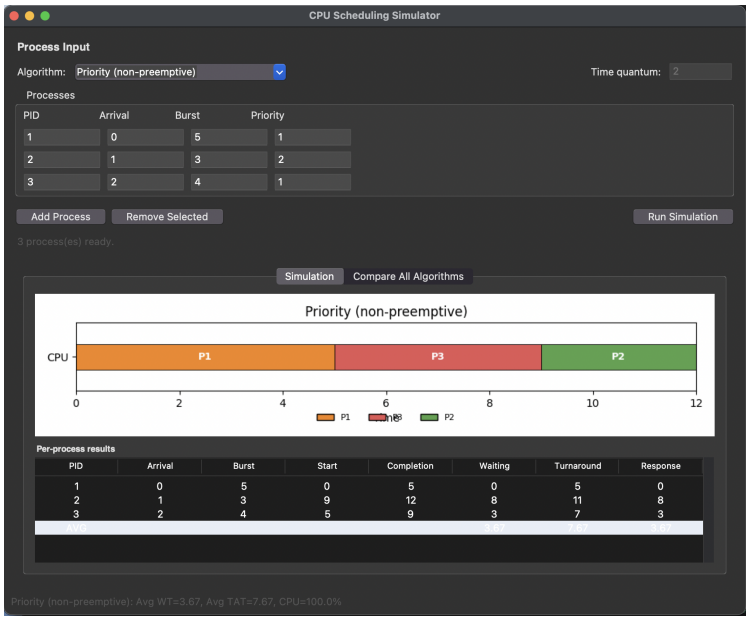


Figure 6. Priority without preemption. P1 is already running when the others arrive. After it, P3 (priority 1) goes before P2 (priority 2). Average waiting time 3.67.

The preemptive priority run (Figure 7) gives the same chart for this input. P3 arrives while P1 is running, but both have priority 1, and equal priority does not cause preemption.

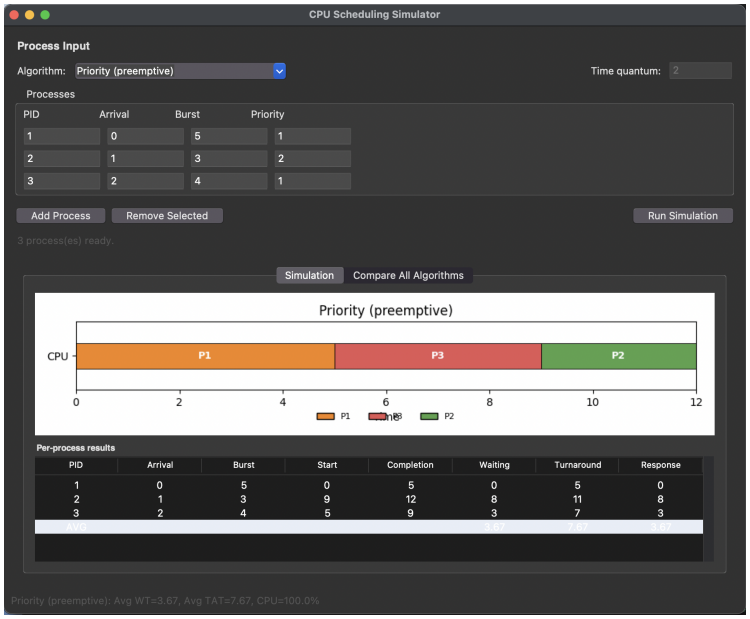


Figure 7. Priority with preemption. Same chart and averages as Figure 6.

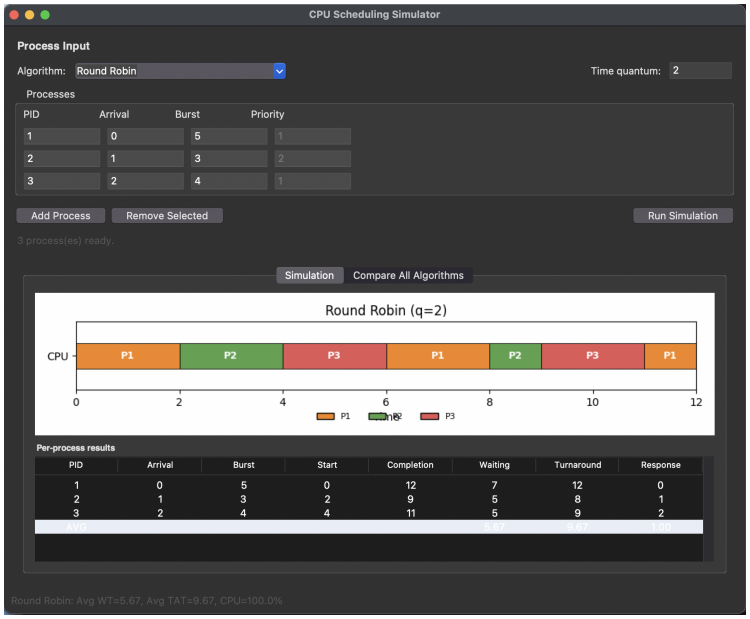


Figure 8. Round Robin with quantum 2. Every process gets the CPU early, so the average response time is the lowest at 1.00. The cost is a higher average waiting time (5.67) and turnaround (9.67).

The Compare All Algorithms tab runs all six on this input and draws the averages side by side.

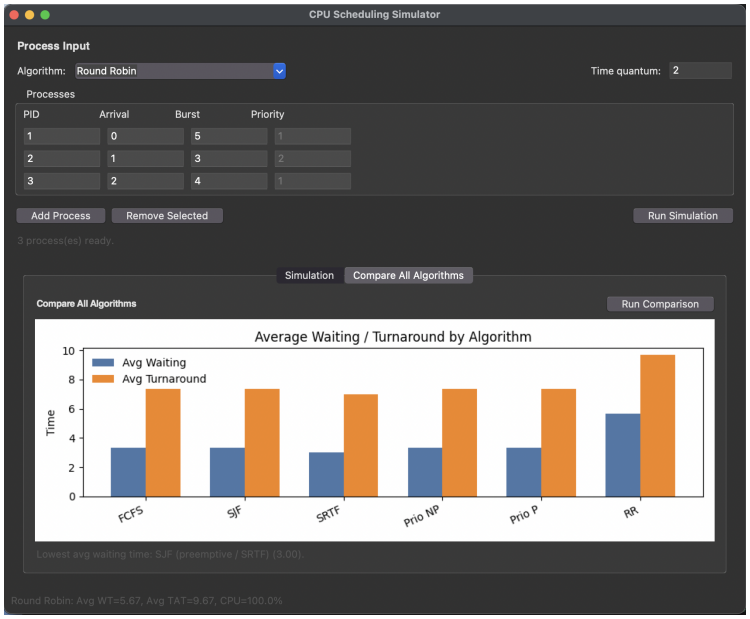


Figure 9. Comparison view. SRTF has the lowest average waiting time (3.00). Round Robin is the highest on both measures for this workload.

The tests cover the same ground without the GUI. The suite has 39 tests, for FCFS, SJF, Priority, Round Robin, the metrics module and edge cases such as an empty input, duplicate PIDs, identical arrival times and idle gaps. All 39 passed when I ran `python3 -m pytest tests/ -v` for this document. The first lines of that run look like this.

```
collected 39 items

tests/test_edge_cases.py::test_empty_process_list[FCFS-<lambda>] PASSED  [  2%]
tests/test_edge_cases.py::test_empty_process_list[SJF-NP-<lambda>] PASSED [  5%]
tests/test_edge_cases.py::test_empty_process_list[SJF-P-<lambda>] PASSED [  7%]
...
============================== 39 passed in 0.04s ==============================
```

12. Future scope

Several things could be added without changing the core design. Multilevel queue and multilevel feedback queue scheduling would be natural next policies, since the textbooks cover them. A context-switch cost could be added as an option, because the current model treats a switch as free. Aging for priority scheduling would show how starvation is avoided. The input form could load processes from a CSV file, and the Gantt chart and results could be exported as an image or a file. An animated step-by-step playback would help in a classroom. Multi-core scheduling is a larger change but follows the same timeline idea with one row per core.

13. Challenges and ethical considerations

Challenges

The hardest part was getting the preemptive algorithms and Round Robin to behave the same way as the textbook answers. Round Robin has no single agreed rule for a process that arrives at the same moment another one is preempted. I followed the common textbook rule of putting the new arrival ahead of the preempted process, and wrote that rule into the code comments and the tests. Ties were the second issue. Without a fixed rule, two valid answers exist for the same input, so the tie-break by PID is applied everywhere.

The third problem was environmental. Python builds without Tkinter are common on macOS, and the GUI cannot start on them. Keeping the scheduling code free of any GUI import meant the tests still run on those builds, and the launch script finds a suitable interpreter for the GUI.

There is also a known visual flaw. In the Gantt chart the legend overlaps the "Time" axis label slightly, as can be seen in the screenshots. It does not affect any result.

Limitations and ethics

The simulator treats time as whole units and assumes a single CPU, no context-switch cost and no input or output waits. Its numbers describe the textbook model, not the behaviour of a real kernel, and it should not be used to draw conclusions about real systems. The document and the program say so plainly.

The project uses no personal data, no network access and no user accounts. The comparison between algorithms is only valid for the workload entered, and a different workload can reverse the ranking. A fair discussion of the results has to say that. The survey in Section 5 cites only works I could confirm exists, and the GitHub projects were judged from their public descriptions, not from running them. No code from those projects was used.

14. Conclusion

The project implements six CPU scheduling policies, shows each result as a Gantt chart with per-process timings, and compares all of them on one workload. The core is separate from the interface and covered by 39 passing tests. On the sample workload the screens match the numbers a student would get by hand, for example an average waiting time of 3.33 for FCFS and 3.00 for SRTF. The simulator is limited to a single CPU and whole time units, and the next steps would be multilevel queues, context-switch cost and file import and export.

References

[1] Silberschatz, A., Galvin, P. B. and Gagne, G. *Operating System Concepts*. Wiley. Chapters on CPU scheduling.

[2] Tanenbaum, A. S. and Bos, H. *Modern Operating Systems*. Pearson. Chapter on processes and scheduling.

[3] Arpaci-Dusseau, R. H. and Arpaci-Dusseau, A. C. *Operating Systems: Three Easy Pieces*. University of Wisconsin-Madison. https://research.cs.wisc.edu/wind/OSTEP

[4] Liu, C. L. and Layland, J. W. Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment. *Journal of the ACM* 20(1), 46-61, January 1973. DOI 10.1145/321738.321743.

[5] Linux kernel documentation, CFS Scheduler (sched-design-CFS). https://www.kernel.org/doc/Documentation/scheduler/sched-design-CFS.rst

[6] Manas-Maahir, OS algorithms visualization. https://github.com/Manas-Maahir/OS-algorithms-visualization-

[7] Azriy-Nazariee, CPU-Scheduling. https://github.com/Azriy-Nazariee/CPU-Scheduling

[8] kitzki, Module4Assign3. https://github.com/kitzki/Module4Assign3

[9] def3r, SchedSim. https://github.com/def3r/SchedSim

[10] Kaushal, R., CPU scheduling simulator. https://rohitkaushal.is-a.dev/cpu-scheduling-simulator/

[11] Kayal, A. CPU Scheduling Simulator (this project). https://github.com/anubhav-kayal/os-lab-project